HROne | Software Engineer Hiring | Contract v2.0

# Engineering Assignment

Employee Attendance & Analytics API (FastAPI + MongoDB)

## 1. Overview

You will build the backend of a small HR attendance system: employees punch in and out, managers correct records, and the business asks questions of the data (who is late, which department is slipping, what does the trend look like). The data lives in MongoDB. You write the service in FastAPI.

This is not a tutorial. The contract is precise, the business rules are written in prose, and the dataset we test with is larger and nastier than the sample we give you. Plausible-looking output is not the same as correct output. Please read everything before you start.

| Part | Guide time | What you do |
|---|---|---|
| A. Review | 45 min | A colleague left a half-finished `app/main.py`. It runs, but it has not been reviewed. Find what is wrong and fix it. Record your findings in `REVIEW.md`. |
| B. Build | 2.5 - 3 h | Implement the remaining endpoints, including three non-trivial MongoDB aggregation pipelines. |
| C. Prove & explain | 30 min | Add an explain endpoint that proves your queries use indexes. Answer five questions in `DECISIONS.md`. |

**Stack:** Python 3.11+, FastAPI, PyMongo or Motor, MongoDB 6.0+ (Atlas free tier or local Docker). **Total guide time:** about 4 to 5 hours.

## 2. What you receive and what you submit

| You receive (candidate kit) | Purpose |
|---|---|
| `PROBLEM_STATEMENT.docx` | This document. |
| `openapi.yaml` | The contract: paths, parameters, status codes, response fields and the business rules R1 - R10. |
| `app/main.py` | The starter application (Part A). |
| `DATA_MODEL.md` | The stored data: collections, field types, rules for seeded data, and nine annotated sample documents. |
| `sample_data/`, `sample_seed.py` | The nine sample documents as MongoDB Extended JSON, and a script that loads them into your database. |
| `REVIEW.md`, `DECISIONS.md` | Templates to fill in. |

| You submit (public Git repository) | Notes |
|---|---|
| `app/main.py` | **All** application code in this one file. |
| `requirements.txt` | At the repository root. Include everything you import. |
| `REVIEW.md`, `DECISIONS.md` | Parts A and C. |
| `README.md` | A few lines: how to run, anything we should know. |
| Do **not** include | `.env`, secrets, a Dockerfile, virtual environments, or data dumps. |

## 3. Ground rules

- Documentation, search engines and your own notes are fine. If you use AI tools, treat them like any other reference: **you** are responsible for every line. Pasting this document into a tool and submitting the result will not go well: the hidden dataset, `REVIEW.md` and the follow-up conversation are designed so that you must understand the code.
- Shortlisted candidates are invited to a live session where you walk through your code, explain choices from `DECISIONS.md`, and make a small change on the spot.
- Submissions are first evaluated automatically. Follow the contract exactly: a wrong field name or status code is a failed test, not a debate.
- Work alone. Submissions are checked for similarity against each other.

## 4. Environment contract

- The app starts with `uvicorn app.main:app --port 8000` from the repository root and must be ready (`GET /health` returns 200) within 20 seconds.
- Read `MONGO_URI` and `MONGO_DB` from environment variables. A local `.env` is fine, but real environment variables must take priority. We inject our own values; never hard-code connection details.
- Create all indexes **at startup**, idempotently. The grader starts your app against a database that already holds the hidden dataset.
- No network access other than MongoDB. Do not call external services.
- The app must handle concurrent requests correctly; we send parallel requests.

## 5. Data model and conventions

MongoDB holds two collections, `employees` and `attendance_logs`. The grader seeds them directly, so your code must read them exactly as `DATA_MODEL.md` describes. That file lists every field and its type and includes nine sample documents. This section summarises what you need to keep in mind before you write any code.

| Convention | What it means |
|---|---|
| Employee identity | `emp_code` identifies an employee everywhere. It is a customer-generated business code (for example `EMP0001`): unique, never changes, and **not** MongoDB's `_id`. `_id` is internal and never appears in a request or response. Attendance records have no id either; a record is addressed by (`emp_code`, `date`). |
| Instants | Every point in time in the API (`punched_at`, `punch_in`, `punch_out`, `created_at`, and the times in `history`) is an integer in epoch **milliseconds**. MongoDB stores them as BSON datetimes (UTC), so you convert at the boundary. Values that look like seconds, floats and strings are rejected with 422. By default PyMongo returns naive datetimes; decide how you will handle that. |
| Calendar values | Dates are `YYYY-MM-DD` (IST), months are `YYYY-MM`, and shift times are `HH:MM` (24-hour, IST), both in the API and in MongoDB. |
| Derived fields | `work_hours`, `late_minutes`, `overtime_minutes` and `half_day` are computed by your API and never sent by a client. They are stored, and analytics read the stored values instead of recomputing them. |
| Gaps in the data | Seeded records may omit `history` or `half_day`; treat them as empty and false. A weekday with **no record at all** is not the same as a record with status `ABSENT`. |
| Indexes | The seeded database has only the default `_id` index. Create the ones you need at startup. |

### Status values

| Status | Meaning | Punch times | Present? | Created by |
|---|---|---|---|---|
| `PRESENT` | Worked on site | `punch_in` required | yes | Punch-in |
| `WFH` | Worked from home | `punch_in` required | yes | Punch-in |
| `ON_DUTY` | Official duty away from the office | `punch_in` required | yes | Punch-in |
| `ABSENT` | Did not work, no approved leave | both null | no | Seeded data, or a correction |
| `LEAVE` | On approved leave | both null | no | Seeded data, or a correction |

### What history is

`history` is the audit trail of **manual corrections only**: what the PATCH endpoint writes. Each correction appends one entry saying when it happened, who made it, why, and the before and after value of every field that actually changed. Punch-in and punch-out never write to it, and entries are never edited or removed. A record that has never been corrected has an empty `history`.

## 6. Business rules

These rules are repeated, with the same numbering, in `openapi.yaml`. They are intentionally not visible in the JSON schemas, so read them carefully; most wrong answers come from skipping one.

| Rule | Statement |
|---|---|
| R1 | **Time zone and attendance date.** Shift times and attendance dates are IST (UTC+05:30). A record's `date` is the IST calendar date of its `punch_in`. Overnight shifts (`shift_end` <= `shift_start`, e.g. 22:00 - 06:00): a punch-in earlier than `shift_end` belongs to the shift that started the previous day, and the shift ends on the next calendar day. Instants are truncated to whole seconds before any calculation, and the truncated value is stored and returned. |
| R2 | **Late.** Grace period 10 minutes. Late only if the punch-in is **strictly more than** 10:00 after `shift_start`; then `late_minutes` = whole minutes (floor) since `shift_start`, not since the grace period ended. Shift 09:30: 09:40:00 gives 0; 09:40:01 gives 10; 10:15:59 gives 45. |
| R3 | **Overtime.** Whole minutes (floor) from `shift_end` to `punch_out`, counted only if that is at least 30; otherwise 0. |
| R4 | **Work hours.** (`punch_out` - `punch_in`) in seconds / 3600, rounded to 2 decimals **half-up**. `null` until punch-out. |
| R5 | **Half day.** After punch-out, `half_day` is true when the **rounded** `work_hours` < 4.50. A half day counts as 0.5 of a present day. |
| R6 | **Presence.** `PRESENT`, `WFH`, `ON_DUTY` count as present. `ABSENT`, `LEAVE` do not. |
| R7 | **Working days.** Monday - Friday, no holiday calendar; counted from `joined_on` for mid-month joiners. `present_days` counts only Mon - Fri records; weekend records still add to late and overtime totals. |
| R8 | **Reported numbers.** 2 decimals half-up (rates: 4 decimals half-up). |
| R9 | **Headcount.** Employees with `joined_on` on or before the end of the period, whether or not they have any logs. |
| R10 | **Pagination.** `page` starts at 1; `page_size` default 20, max 100; `total` always reflects the filters. |

## 7. The API at a glance

The full contract, including every status code, is in `openapi.yaml`. "Starter" means a version exists in the starter file (and may contain defects); "You" means you write it.

| | Endpoint | Summary | Where |
|---|---|---|---|
| GET | `/health` | 200 only when MongoDB answers; else 503 | Starter |
| POST | `/employees` | Create; 409 on duplicate `emp_code`; 422 on invalid input | Starter |
| GET | `/employees` | Filter by department; paginated; sorted by `emp_code` | Starter |
| POST | `/attendance/punch-in` | Creates the day's record (R1, R2); status `PRESENT`, `WFH` or `ON_DUTY`; race-safe | Starter |
| GET | `/attendance` | Filter by employee / dates / status; paginated; fast at 100k docs | Starter |
| POST | `/attendance/punch-out` | Computes work hours, overtime, half day (R3 - R5); works across midnight | You |
| PATCH | `/attendance/{emp_code}/{date}` | Regularize one day's record: recompute derived fields, append to `history` | You |
| GET | `/analytics/employees/{emp_code}/monthly` | Monthly summary for one employee (aggregation) | You |
| GET | `/analytics/departments/summary` | Department summary for a month (aggregation, cross-collection) | You |
| GET | `/analytics/leaderboard/late` | Late-comers ranking with ties (aggregation, window function) | You |
| GET | `/analytics/departments/{department}/trend` | Daily series with gap filling and a 7-day moving average | You |
| GET | `/admin/explain/{endpoint}` | MongoDB `explain` output for an endpoint's main query | You |

## The three hard endpoints

These are where the exercise is decided. The contract defines the behaviour; working out how to get a database to do it is your job.

- **Department summary.** Logs reference employees by `emp_code`, but the department lives on the employee. Headcount must include people with no logs and exclude people who have not joined yet. The average is over records, not an average of averages.
- **Late leaderboard.** Ties share a rank and the next rank is skipped (1, 2, 2, 4). `limit` is applied after ranking. Tied rows are ordered by `emp_code`. Sorting, limiting and using the array index as the rank is the obvious approach, and it is wrong.
- **Department trend.** Exactly one row per calendar day in the range, including weekends and days with no records, each with a moving average over the preceding window. The first rows have a shorter window. The work is done in the database, not in a Python loop.

## 8. Part A: review the starter

Read `app/main.py` as if a teammate had opened a pull request. It runs and the happy path works. There are **more than ten** defects, spread across the helper functions and the endpoints. Some are visible within seconds; some only show up under concurrency, at scale, or with particular inputs.

Fix them, and list each one in `REVIEW.md` using the template: where it is, what is wrong, how you would notice it (an input, a test, or a symptom), and how you fixed it. Also mention anything you examined and decided was fine. A short, precise list with concrete failing examples is better than a long vague one.

The helper functions in the starter are there to be reused. Whether you should reuse them as they are is part of the exercise.

## 9. Part B: build the rest

- Implement every endpoint marked "You" in section 7, exactly as `openapi.yaml` describes.
- Analytics endpoints must be implemented as MongoDB aggregation pipelines, not by fetching documents and looping in Python.
- Concurrency: two simultaneous punch-ins for the same employee and date yield exactly one 201 and the rest 409. Two simultaneous punch-outs yield exactly one 200. Concurrent regularizations must not silently lose each other's history entries.
- Return correct status codes: 404 for missing resources (an unknown employee, or no record for that date), 409 for conflicts, 422 for validation failures (including a bad timestamp). A 500 is always a failed test.
- Everything must be fast on a collection of about 100,000 attendance records. List and analytics endpoints must be backed by indexes.

**Test your own work.** We hand you only a tiny sample. Build your own test data with the awkward cases the rules describe: employees with no logs, ties, weekend records, a mid-month joiner, an overnight shift, a missing weekday. A pass on the sample is not a pass.

## 10. Part C: prove it and explain it

### The explain endpoint

Implement `GET /admin/explain/{endpoint}` where `endpoint` is one of `attendance_list`, `employee_monthly`, `department_summary`, `late_leaderboard`, `department_trend`. It runs MongoDB's `explain` with `executionStats` verbosity on that endpoint's main query or pipeline, **built the same way the real endpoint builds it** (same filters, sort and pagination), and returns the raw output. It accepts the same query parameters as the endpoint it explains. We check that the plan uses an index scan (`IXSCAN`) and contains no collection scan (`COLLSCAN`), against the 100,000-record dataset.

### DECISIONS.md

Answer the five questions in the template in your own words, about 150 to 200 words in total: which indexes you created and why; what exactly happens when two identical punch-ins arrive at the same instant; what your leaderboard returns for a tie at the cutoff; how your department summary counts employees with no logs; and what you would change for 100 times the data. We start the follow-up conversation from this file.

## 11. How we evaluate

| Area | Weight | How it is checked |
|---|---|---|
| Review (Part A) | 10% | `REVIEW.md` against the known defects; the fixes themselves are exercised by the automated tests. |

| Area | Weight | How it is checked |
|---|---|---|
| CRUD, rules and concurrency | 30% | Automated tests on status codes, R1 - R5, pagination, race conditions. |
| Analytics correctness | 30% | Automated comparison against expected results on the hidden dataset (ties, no-log employees, weekends, joiners, overnight shifts). |
| MongoDB usage and speed | 15% | Explain endpoint plans, indexes, and how the endpoints behave on a dataset of about 100,000 attendance records. |
| Decisions and code quality | 15% | `DECISIONS.md`, readability, structure inside a single file, input validation. |

We do not publish the pass mark. Strong automated results lead to a live walkthrough, which is where the final decision is made. Partial submissions are welcome: submit what you have, and say in `README.md` what is missing.

## 12. Submission checklist

- ☐ `uvicorn app.main:app --port 8000` starts from a clean clone after `pip install -r requirements.txt`.
- ☐ All indexes are created at startup; no manual steps.
- ☐ `MONGO_URI` and `MONGO_DB` are read from the environment; nothing is hard-coded.
- ☐ `REVIEW.md` and `DECISIONS.md` are filled in.
- ☐ No `.env`, secrets, Dockerfile or virtual environment are committed.
- ☐ The repository is public, and its link is submitted by the deadline and in the way described in the email we sent you.